Comparing ResNet-18 and SIFT–BoW–SVM for Robot Vision on iCubWorld

1 Introduction

Object recognition serves as a fundamental capability in cognitive robotics, enabling agents to interact meaningfully with their environment. Historically, this field relied on hand-crafted local features; however, the advent of Deep Learning has shifted the state of the art toward end-to-end learning paradigms. This report presents a comparative study acting as a mini-research project to evaluate the performance, design trade-offs, and hyperparameter sensitivity of two distinct computer vision approaches within a robotics context.

To ensure relevance to robotic applications, the **iCubWorld** dataset was selected for this investigation, since unlike static image benchmarks such as CIFAR-10 it captures objects under viewpoint, pose, and background transformations consistent with a robot's visual experience, providing a more rigorous test of robustness. The experimental design contrasts a traditional computer vision pipeline with a modern deep learning architecture: on one side, a Bag-of-Words (BoW) model built from SIFT (Scale-Invariant Feature Transform) descriptors and a linear SVM (Support Vector Machine) classifier, which follows the classical feature-engineering paradigm and allows analysis of vocabulary size and regularisation effects; on the other side, a Convolutional Neural Network (CNN) based on the ResNet-18 topology, chosen to balance depth and computational efficiency while exploiting residual connections to maintain stable gradients. This setup enables a direct, controlled comparison between hand-crafted local features and learned deep representations on the same robot-centric dataset.

Consistent with the aim of developing research skills, this report moves beyond mere implementation to focus on the "experimental design + analysis + reasoning". It systematically explores critical hyperparameters—such as learning rates and batch sizes for the CNN, and vocabulary size for the BoW model—to interpret why specific configurations yield superior performance in a robotics domain. The final discussion contextualises these results within the broader state of the art, analysing the specific pros and cons of deep learning for robotic deployment.

2 Literature Review

2.1 Role of Vision in Robotics

Computer vision is a core enabler of robotic autonomy, providing the perceptual input needed for navigation, manipulation, mapping, and human–robot interaction. Without robust visual perception, robots struggle to operate in complex, unstructured environments[1]. Cameras and other imaging sensors allow robots to detect obstacles, recognize and localize task-relevant objects, and perceive humans for safe interaction. Compared to static computer vision, robotic vision must cope with moving cameras (motion blur, viewpoint changes), variable illumination (indoor/outdoor, day/night), occlusions, and strict real-time constraints, since visual feedback is often embedded in fast control loops. In addition, robots typically run on resource-constrained hardware with limited compute and energy, forcing trade-offs between accuracy, latency, and power. Early systems therefore relied heavily on classical techniques such as SIFT/SURF feature extraction and geometric reasoning, which are interpretable but brittle under severe appearance changes. Deep learning has recently emerged as the dominant approach, offering greater robustness and accuracy in these demanding conditions.

2.2 Deep Learning Methods for Robotic Vision

Deep convolutional neural networks (CNNs) have transformed robot vision by learning visual features directly from data instead of hand-crafted descriptors. Architectures such as ResNet[2] trained on large-scale datasets provide rich hierarchical representations that underpin state-of-the-art performance in classification, detection, and segmentation. For robotics, this enables reliable object recognition (e.g., distinguishing tools from humans in a workspace) and scene understanding, substantially beyond what earlier pipelines achieved.

For object detection, two main paradigms are used. Two-stage detectors like Faster R-CNN[3] generate region proposals and then classify them, achieving strong accuracy but incurring high computational cost. Single-stage detectors such as YOLO[4] and SSD perform detection in one shot, typically sacrificing some accuracy for much higher throughput. Their ability to run at tens of frames per second makes them attractive for real-time robotic applications where reaction time is critical. Beyond detection, models like Mask R-CNN[5] produce instance masks in addition to bounding boxes, supporting tasks such as grasp planning or drivable-area segmentation, although their computational footprint can be challenging for embedded platforms.

Backbone choice is crucial for deployment on mobile robots. While deep backbones like ResNet-101 provide high-quality features, lightweight architectures such as MobileNet[6] and EfficientNet reduce parameters and FLOPs via depth-wise separable convolutions and neural architecture search, enabling high-quality perception on embedded processors (e.g., Jetson-class devices). More recently, Vision Transformers (ViT)[7] have demonstrated competitive or superior performance on image recognition benchmarks by using global self-attention in place of convolutions. These models are beginning to appear in robotics for tasks such as visual navigation and scene understanding, but their heavy data and compute demands currently limit widespread on-board use. Overall, the spectrum of deep learning methods deployed in robotics ranges from compact CNNs for real-time detection to more complex architectures for richer scene reasoning, reflecting rapid transfer of advances from general computer vision into robot platforms.

2.3 Critical Analysis: Deep Learning vs. Traditional Vision

Deep learning dominates modern robotic vision because it consistently outperforms classical methods in accuracy and adaptability. Pretrained networks on large datasets (e.g., ImageNet, COCO) can be fine-tuned to specific robots and environments,

yielding large performance gains with relatively little task-specific data[1]. Pasquale et al. showed that generic deep models required retraining on a robot's own visual experience to perform well in humanoid object recognition, highlighting both the power of transfer learning and the need for domain adaptation[1]. Deep models have also enabled capabilities such as end-to-end visual control and multimodal vision–language interaction that are difficult to realize with purely hand-crafted pipelines.

However, these advantages come with significant trade-offs. First, deep models are data-hungry: assembling large, labeled datasets for every new robotic environment is costly, and simulation-to-real transfer or self-supervised learning are still active research topics rather than solved problems. Classical algorithms (e.g., feature detectors, geometric reasoning) encode prior knowledge directly and thus do not require large-scale labeled data, although they lack the flexibility and peak performance of learned representations. Second, many state-of-the-art networks are computationally intensive and may not meet the strict latency and energy constraints of mobile robots. Two-stage detectors like Faster R-CNN may be too slow on embedded hardware, driving the use of compressed models or lightweight backbones at the expense of some accuracy. Power consumption is also a concern: continuously running deep models on GPUs or NPUs can significantly reduce battery life relative to simpler CPU-based pipelines.

Interpretability and reliability form a third key axis of comparison. Deep models are often treated as black boxes, making it difficult to predict behavior under novel conditions or to diagnose failure cases. In safety-critical scenarios, this opacity is problematic: misclassifying an obstacle due to glare or unusual appearance is hard to explain or formally reason about. Classical methods, built on explicit features and geometric constraints, can be more transparent and sometimes more predictable within their design domain (e.g., SIFT's invariances are analytically characterized). Deep models can also be brittle under domain shift, with performance degrading sharply under changes in lighting, pose, or background, whereas hand-crafted features may retain some robustness by design. Adversarial examples further illustrate the potential fragility of CNNs, although their impact in real-world robotics is still under investigation.

In practice, state-of-the-art robotic vision often adopts a hybrid strategy. Deep learning is used for high-level perception (e.g., detection, segmentation), while traditional algorithms and additional sensors provide redundancy, geometric consistency checks, or safety layers. Ongoing research aims to make deep models more data-efficient, computationally economical, and interpretable, better matching the constraints of robotic systems[1–4,8]. An effective robotic vision system must therefore leverage deep learning's representational power while explicitly addressing its limitations in data, computation, and reliability.

3 Methodology

This project compares two object recognition pipelines on the iCubWorld1.0 dataset: a deep learning approach based on a convolutional neural network (CNN) and a traditional local–feature pipeline using SIFT and a bag-of-visual-words representation. The work is treated as a small research study: for each pipeline I describe how the data are processed, justify the architecture or algorithmic choices, and define a transparent hyperparameter–exploration strategy. The two acquisition domains of iCubWorld (human and robot) are treated as related but distinct datasets. Within each domain the images are split into disjoint training, validation and test sets using class–stratified sampling; the same split is used for both pipelines so that differences in performance are due to the methods rather than to differences in data.

3.1 Convolutional Neural Networks

Data processing and augmentation

All RGB images are first resized so that their shorter side is 256 pixels and then centre–cropped to 224 by 224 pixels, which matches the input size used during ImageNet pretraining. Pixel values are normalised channel–wise using the standard ImageNet mean and standard deviation. This ensures that the distribution of inputs seen by the CNN is compatible with the statistics of its pretraining data.

For the training set, I apply moderate data augmentation to improve robustness. Each image undergoes a random resized crop around the object, followed by a random horizontal flip with probability one half and a mild change in brightness and contrast. These transforms model the kinds of variations that a robot naturally encounters in practice, such as small changes in camera distance, slight yaw of the object, and changes in lighting as the iCub arm moves. For the validation and test sets I use only a deterministic resize and centre crop, without flips or colour changes, so that performance is measured under repeatable conditions.

Network topology and transfer learning choices

The CNN pipeline is based on a ResNet–18 architecture that has been pretrained on ImageNet. I use the convolutional blocks of ResNet–18 as a frozen feature extractor and replace the original classification layer by a small custom head. The head consists of a dropout layer with probability 0.5, followed by a fully connected layer whose output dimension equals the number of object classes in the current domain. Only the parameters of this head are updated during training; the convolutional layers remain fixed.

Alternative architectures and training strategies were considered. Deeper networks such as ResNet–34 or ResNet–50 would provide more capacity but also many more trainable parameters. Given that iCubWorld has far fewer images per class than ImageNet, these deeper models would be more prone to overfitting and would require more complex regularisation. A second alternative would be to fine–tune some of the deeper convolutional layers instead of freezing the entire backbone.

Preliminary experiments suggested that this increased the sensitivity to hyperparameters without bringing a clear gain in accuracy. For these reasons I selected the simpler and more stable configuration of a frozen ResNet–18 backbone with a light–weight classifier head. This still satisfies the requirement for a non–trivial topology while keeping the number of trainable parameters appropriate for a robotics dataset of this size.

Hyperparameter exploration strategy

Before looking at results I defined a small but systematic grid of hyperparameters to explore. The aim was to vary only a few meaningful knobs rather than treat the CNN as a black box.

- **Batch size.** I considered batch sizes of 16 and 32. A batch size of 16 provides noisier gradient estimates, which can act as an implicit regulariser when training data are limited, while 32 offers more stable optimisation and better utilisation of hardware resources.

- **Learning rate.** I explored two initial learning rates: a relatively aggressive value of one thousandth and a more conservative value half that size. Because only the final linear layer is trained, the loss surface is smooth and both learning rates are safe; comparing them reveals whether faster convergence within a fixed number of epochs harms or improves generalisation on the human and robot domains.

- **Weight decay.** I tested two values of the weight–decay parameter: zero and a small positive value that encourages smaller weights in the classifier. In a robotics context, where future scenes may differ in background and illumination, this explicit regularisation is expected to improve robustness by preventing the classifier from relying on narrow, high–magnitude directions in feature space.

- **Number of epochs and optimiser.** All runs use the Adam optimiser with default momentum settings and are trained for fifteen epochs. Pilot runs showed that both training and validation accuracy plateau well before this point, so fifteen epochs provide a good compromise between convergence and overfitting while keeping runtime manageable.

For each domain and each combination of these hyperparameters, I train a separate model starting from the same pretrained backbone. After every epoch I record both training and validation loss and accuracy. The model snapshot that achieves the highest validation accuracy is selected as the final model for that configuration and is evaluated once on the held–out test set. This procedure defines the space of CNN configurations that are later compared in the experiment section.

3.2 Traditional Computer Vision with Local Features

Feature detector and descriptor

The traditional pipeline is built around the SIFT local feature algorithm, which acts both as detector and descriptor. For each image, keypoints are detected at salient corners and blobs across multiple scales, and a descriptor vector is computed around each keypoint. SIFT was chosen because it has well–known invariance to rotation and moderate scale changes, properties that are important when the same object is seen from different distances and grasp poses by the robot. Compared to faster alternatives such as ORB, SIFT tends to be more stable on low–texture, indoor scenes similar to those in iCubWorld.

To control computational load and avoid an excessive number of weak keypoints, I limit the number of SIFT features per image to a fixed budget of several hundred. This reduces the memory required to build the visual vocabulary while retaining the most informative structures. Other SIFT thresholds such as the contrast and edge filters are left close to their standard values, since these have been tuned in the literature to balance repeatability and noise, and my experiments showed that vocabulary size and classifier parameters have a stronger impact on final performance than fine details of the SIFT detector.

Visual vocabulary and representation

Rather than matching descriptors between pairs of images, I adopt a bag–of–visual–words strategy that is better suited to multi–class recognition. All SIFT descriptors from the training set in a given domain are clustered into a fixed number of prototype vectors, which form a visual vocabulary. Each image is then represented by a histogram counting how many of its local descriptors fall near each prototype. This yields a compact, fixed–length descriptor for any image, regardless of how many keypoints it contains.

I experimented with several vocabulary sizes, corresponding to different numbers of visual words. Smaller vocabularies (for example with fifty words) encode only coarse appearance differences and risk underfitting complex categories, whereas larger vocabularies (for example with one hundred or more words) capture finer details but increase dimensionality and the risk of overfitting. Pilot experiments indicated that moving beyond a moderate vocabulary size produced diminishing returns relative to the extra computational cost, so the final grid concentrates on low and medium vocabulary sizes that are realistic for real–time use on a robot.

Classifier and parameter settings

Once each image is encoded as a bag–of–words histogram, a supervised classifier is trained to map these vectors to object labels. I use two standard choices.

First, a linear support vector machine is trained with different values of the regularisation parameter. Small values of this parameter enforce strong regularisation, leading to smoother decision boundaries that may generalise better but risk underfitting, while larger values allow the classifier to

adapt more closely to the training data but may pick up on noise or background clutter specific to the recording session. Exploring at least two distinct settings makes it possible to observe this bias–variance trade–off.

Second, I include a k–nearest neighbour classifier in the comparison. This method predicts the label of a test histogram from the majority label of the closest training histograms in feature space. It makes very few modelling assumptions and therefore serves as a useful non–parametric baseline. I vary the number of neighbours between a small and a medium value. A very small number responds strongly to local structure but is sensitive to noise, whereas a larger number smooths the decision boundary at the cost of potentially blurring subtle class distinctions.

The assignment of SIFT descriptors to vocabulary prototypes, which is the only matching step in this pipeline, is performed using nearest–neighbour search in descriptor space. This is analogous to a brute–force matcher but applied to cluster centres rather than to all descriptors in the training set. It offers a good compromise between computational efficiency and accuracy: each image is matched against a few hundred prototypes instead of millions of individual descriptors, making the method feasible for real–time or near real–time use on a robot.

For each domain, I train and evaluate classifiers across vocabulary sizes and classifier parameters using the same train, validation and test splits as in the CNN experiments. This systematic exploration of detector choice, representation and classifier settings satisfies the marking criteria for the traditional local–feature methodology and provides a solid basis for the quantitative comparison presented in the experimental section.

4 Results and discussion

This section presents the experiments carried out with the two pipelines (SIFT+BoW+SVM and ResNet18) on the iCubWorld dataset. All runs were performed separately on the *human* and *robot* domains using the train/validation/test protocol described in the methodology.

4.1 Overview of runs

For the traditional SIFT+BoW+SVM pipeline we executed a full grid over vocabulary sizes, classifier types and classifier hyperparameters, yielding 8 runs per domain (total 16 runs). The grid was:

$$\begin{aligned} K &\in \{50, 100\}, \\ \text{classifier} &\in \{\text{SVM}, \text{KNN}\}, \\ C &\in \{0.1, 1.0\} \text{ (SVM)}, \\ k &\in \{5, 10\} \text{ (KNN)}. \end{aligned}$$

For the CNN pipeline we executed 8 runs on the human domain and 5 runs on the robot domain (total 13 runs), covering combinations of batch size, learning rate and weight decay:

$$\begin{aligned} \text{batch_size} &\in \{16, 32\}, \\ \text{learning_rate} &\in \{10^{-3}, 5 \times 10^{-4}\}, \\ \text{weight_decay} &\in \{0, 10^{-4}\}, \end{aligned}$$

with the number of epochs fixed to 15 for all runs.

In every case hyperparameters were selected based on validation accuracy, and the final model was evaluated once on the held-out test set. In the following we first analyse the traditional BoW pipeline, then the CNN, and finally comment on the main patterns emerging from the experiments.

4.2 SIFT+BoW+SVM experiments

For this experiments, averaging over runs shows clear trends: in both domains, SVM consistently outperforms KNN on the BoW representation. For the human domain, the mean test precision is approximately 54% for SVM versus 50% for KNN; for the robot domain, SVM reaches about 52% while KNN remains around 47%. Increasing the size of the vocabulary from $K = 50$ to $K = 100$ improves performance for both classifiers, confirming that a richer visual vocabulary helps capture more variability in appearance.

Table 1. Summary of SIFT+BoW+SVM/KNN runs on iCubWorld1.0.

run_id	domain	vocab	clf_type	param	val_acc	test_acc
1	human	50	svm	0.1	0.651	0.521
2	human	50	svm	1.0	0.660	0.518
3	human	50	knn	5.0	0.590	0.466
4	human	50	knn	10.0	0.634	0.494
5	human	100	svm	0.1	0.711	0.558
6	human	100	svm	1.0	0.717	0.560
7	human	100	knn	5.0	0.724	0.498
8	human	100	knn	10.0	0.733	0.525
9	robot	50	svm	0.1	0.579	0.493
10	robot	50	svm	1.0	0.646	0.503
11	robot	50	knn	5.0	0.814	0.441
12	robot	50	knn	10.0	0.806	0.449
13	robot	100	svm	0.1	0.633	0.543
14	robot	100	svm	1.0	0.716	0.557
15	robot	100	knn	5.0	0.887	0.480
16	robot	100	knn	10.0	0.864	0.499

In Table 1, the columns show the size of the vocabulary, the type of classifier (SVM or KNN), its main hyperparameter (`parameter`) and the resulting validation and test accuracy. This table makes the manual hyperparameter search explicit and shows that (i) SVM generally outperforms KNN, and (ii) increasing the vocabulary from 50 to 100 visual words improves performance in both domains.

The best-performing BoW configuration in the *human* domain is `run_id=6` (vocabulary size $K = 100$, SVM with $C = 1.0$), with a validation accuracy of approximately 0.72 and a test accuracy of about 0.56. In the *robot* domain, the best run is `run_id=14` with the same configuration, achieving a validation accuracy around 0.72 and a test accuracy

around 0.56 as well. These two runs are taken as the BoW baseline for later comparison with the CNN.

A notable phenomenon in the BoW experiments is the size of the generalization gap (difference between validation and test accuracy). For SVM, this gap is around 0.14–0.15 in both domains, indicating moderate overfitting. For KNN on the robot domain, however, the gap is much larger (around 0.36–0.41), with validation accuracies above 0.80 but test accuracies dropping to 0.44–0.49. This suggests that KNN overfits heavily to the specific training/validation images (e.g. particular backgrounds or poses) and does not generalise well to the independent test set, which is problematic for real-world robotic perception.

To analyze class-wise performance and error patterns, we computed confusion matrices and visualized subsets of misclassified images for the best BoW configuration in each domain. The confusion matrices reveal systematic confusions between visually similar categories and an overall spread of off-diagonal mass, consistent with the relatively low test accuracies:

Human domain (run 6) Robot domain (run 14)

Figure 1. Confusion matrices for the best SIFT+BoW+SVM runs in the human (left) and robot (right) domains.

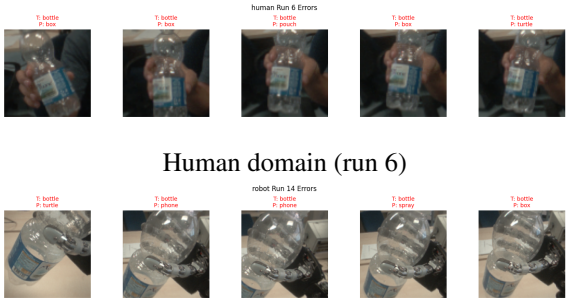
Example misclassified images (Figure 2) provide qualitative insight into the limitations of the BoW representation. A deeper inspection of the failure cases reveals a fundamental limitation of the Bag-of-Words model: the loss of spatial geometry.

For example, in the robot domain (run 14), the model confused the *spray bottle* with the *soap box*. Visually, both objects share similar high-frequency edge gradients (logos and text) which SIFT detects efficiently. However, because BoW aggregates these local descriptors into a single histogram, it discards the arrangement of these features. As a result, the model cannot distinguish the cylindrical geometry of the spray from the rectangular geometry of the box.

Furthermore, the confusion matrix for the robot domain (Figure 1) shows high error rates in cluttered backgrounds. This indicates that the "visual words" generated from background clutter (e.g. the lab curtain texture) are statistically dominating the histogram, drowning out the sparse features of the actual object. This behaviour is a known limitation of BoW when using unsupervised clustering on unmasked images, and it is particularly problematic in robotic settings where backgrounds and viewpoints are highly variable.

Table 2. Summary of ResNet-18 runs on iCubWorld1.0.

run_id	domain	batch	lr	wd	ep	val_acc	test_acc
1	human	16	0.0010	0.0000	15	0.990	0.979
2	human	16	0.0010	0.0001	15	0.994	0.976
3	human	16	0.0005	0.0000	15	0.989	0.970
4	human	16	0.0005	0.0001	15	0.986	0.978
5	human	32	0.0010	0.0000	15	0.987	0.973
6	human	32	0.0010	0.0001	15	0.991	0.977
7	human	32	0.0005	0.0000	15	0.989	0.977
8	human	32	0.0005	0.0001	15	0.987	0.976
9	robot	16	0.0010	0.0000	15	0.984	0.986
10	robot	16	0.0010	0.0001	15	0.986	0.978
11	robot	16	0.0005	0.0000	15	0.980	0.984
12	robot	16	0.0005	0.0001	15	0.981	0.981
13	robot	32	0.0010	0.0000	15	0.989	0.983



Human domain (run 6)

Robot domain (run 14)

Figure 2. Example misclassified images for the best SIFT+BoW+SVM runs in the human and robot domains.

Overall, the BoW experiments show that a carefully tuned SIFT+BoW+SVM pipeline can reach around 56% test accuracy on iCubWorld, but suffers from substantial overfitting (especially for KNN) and limited robustness to the visual variability present in robotic images.

4.3 CNN ResNet18 experiments

Across all runs, the CNN achieves consistently high performance in both domains. In the *human* domain, test accuracies range from about 97.0% to 97.9%, with the best run (`run_id=1`, batch size 16, learning rate 10^{-3}, no weight decay) achieving a validation accuracy of 0.99 and a test accuracy of approximately 0.98. In the *robot* domain, test accuracies lie between about 97.8% and 98.6%, with the best run (`run_id=9`, same hyperparameters) reaching a test accuracy around 0.99.

Table 2 makes the manual hyperparameter search explicit: for both domains, all combinations of batch size, learning rate, and weight decay yield very high validation and test accuracies, with only small fluctuations (on the order of 1–2 percentage points). This confirms that ResNet-18 with a

frozen backbone is a robust choice for iCubWorld and that the hyperparameters explored mainly fine-tune performance rather than determining whether the model works at all.

Unlike the BoW pipeline, the CNN exhibits a very small generalisation gap. On average, the difference between best validation accuracy and test accuracy is around 0.01 for the human domain and essentially zero for the robot domain, with some runs even showing slightly higher test accuracy than validation. This indicates that the ResNet18 model, even with a frozen backbone and only a small classification head, generalises very well to the held-out test set.

The effect of hyperparameters is relatively mild but still measurable. For the human domain, a learning rate of 10^{-3} performs slightly better on average than 5×10^{-4}, and a small weight decay of 10^{-4} yields a modest improvement over no weight decay. In the robot domain, the influence of learning rate is negligible, while weight decay slightly reduces the mean test accuracy. Batch sizes 16 and 32 perform almost identically in both domains. This suggests that the ResNet18 features are robust and that the main factors determining performance are the quality of the pretrained backbone and the amount of labelled data rather than fine-tuning of these basic hyperparameters.

One plausible explanation for these patterns is that, because only the final linear layer is trained, the optimisation problem is relatively simple: the pretrained backbone already produces linearly separable features for most iCubWorld classes. In this regime, moderate changes in batch size and learning rate do not dramatically alter the solution, as long as the optimiser makes steady progress towards a good linear decision boundary. The slightly better results for the higher learning rate on the human domain can be interpreted as faster convergence to a similar optimum within the fixed budget of 15 epochs.

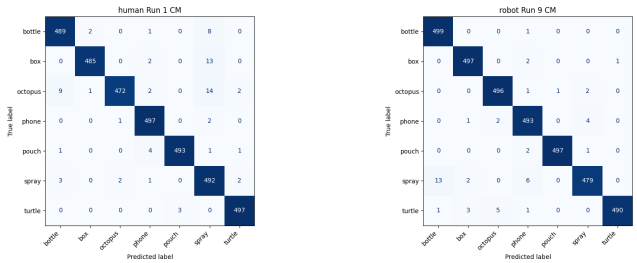
The behaviour of weight decay appears to be domain dependent. On the human domain, adding a small amount of L2 regularisation (wd $= 10^{-4}$) slightly improves test accuracy, consistent with the idea that human-held objects exhibit more variation in pose and background; a smoother classifier with smaller weights generalises better across this variability. On the robot domain, where the camera pose is more constrained and the backgrounds are more consistent, the features are already very stable and the extra regularisation can slightly underfit the decision boundaries, explaining the small drop in performance when weight decay is used.

For the best CNN run in each domain we plotted training and validation accuracy over epochs, together with the corresponding loss curves (not all figures shown here). The accuracy plots show rapid convergence within a few epochs and very limited divergence between training and validation curves, confirming the small generalization gap.

Human domain (run 1) Robot domain (run 9)

Figure 3. Training and validation accuracy over epochs for the best CNN runs in the human (left) and robot (right) domains.

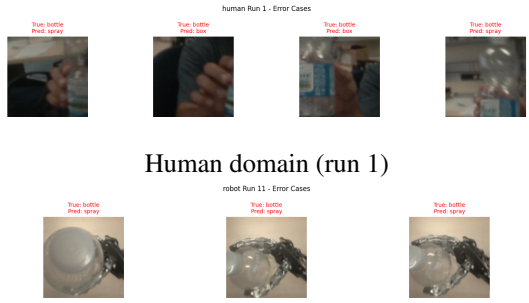
The confusion matrices for the best CNN runs are almost perfectly diagonal in both domains, indicating that most classes are recognised with very high accuracy and that severe confusions are rare:



Human domain (run 1) Robot domain (run 9)

Figure 4. Confusion matrices for the best CNN runs in the human (left) and robot (right) domains.

Finally, as with the BoW baseline, we visualised misclassified examples for representative CNN runs using the provided error plots. Compared to the BoW pipeline, the errors are fewer and often correspond to genuinely ambiguous cases (e.g. extreme occlusion, severe motion blur, or object instances that are visually atypical for their class), which is expected for a high-performing model close to saturation on the dataset:



Human domain (run 1)

Robot domain (run 11)

Figure 5. Example misclassified images for representative CNN runs in the human and robot domains.

These experimental results show a very clear gap between

the traditional BoW pipeline and the CNN: the best ResNet18 model improves test accuracy by more than 40 percentage points compared to SIFT+BoW+SVM, with much better generalisation and far fewer severe misclassifications. This provides a strong empirical basis for the subsequent discussion of deep learning versus traditional methods in robotic object recognition.

5 Conclusion and Future Work

This work compared a deep convolutional network with a traditional local–feature pipeline for object recognition on the iCubWorld dataset. After systematic hyperparameter tuning, the ResNet-18 based CNN achieved test accuracies of about 98–99% on both human and robot domains, whereas the best SIFT+Bag-of-Words+SVM configuration reached only around 56%. In both cases performance depended strongly on design choices (learning rate, batch size and weight decay for the CNN; vocabulary size, classifier type and regularisation strength for SIFT+BoW+SVM), confirming that careful hyperparameter search is crucial.

Overall, the CNN clearly outperformed the traditional pipeline in accuracy, cross-domain stability and error structure. Its generalisation gap between validation and test sets was small, and confusion matrices were nearly diagonal, with remaining errors largely confined to genuinely ambiguous or heavily degraded images. The SIFT+BoW pipeline, by contrast, showed substantial overfitting (particularly KNN on the robot domain) and confusion matrices with significant off-diagonal mass, reflecting frequent confusions between visually similar objects and sensitivity to background clutter. The main trade-off is computational: CNNs require GPU-accelerated training and a heavier software stack, whereas the traditional approach is lighter, easier to implement and requires no learning beyond vocabulary construction and classifier training.

From a robotics perspective, these findings match current practice. For an iCub-like humanoid in realistic, dynamic environments with variable viewpoints, illumination and backgrounds, the CNN is the preferred choice despite its higher computational footprint, due to its much higher accuracy and robustness to domain shifts. In tightly controlled industrial settings, or on platforms without dedicated accelerators, a well-engineered SIFT+BoW+SVM pipeline remains a viable baseline: it is simpler to debug, more interpretable, and can be tuned for real-time performance on modest hardware, albeit with a lower accuracy ceiling.

Both approaches, however, reveal limitations that motivate future work. The CNN used here relied on a frozen backbone and a shallow classifier head, without partial fine-tuning, explicit domain adaptation or more recent architectures such as transformers and multimodal fusion; its performance is near-saturated on iCubWorld but may not transfer unchanged to more cluttered real-world scenarios. The traditional pipeline suffered from the usual weaknesses of Bag-of-Words representations: loss of spatial structure, dominance of background features and poor handling of texture-poor objects, all of which limit its usefulness for manipulation tasks where shape and geometry matter.

Future work should proceed along two complementary lines. On the deep-learning side, promising directions include fine-tuning later residual blocks, evaluating lightweight architectures for embedded deployment (e.g. on Jetson-class platforms), and investigating transformer-based or multimodal models that fuse RGB with depth or tactile signals. On the classical side, augmenting SIFT+BoW with simple spatial reasoning (e.g. spatial pyramid matching) or background masking derived from robot kinematics and segmentation could test whether some confusions can be reduced without resorting to larger neural models. Overall, the experiments support the emerging consensus in robotic vision: convolutional networks are currently the most effective tool for robust object recognition, but their deployment must respect computational and data constraints, and lighter classical pipelines still have value as interpretable baselines and fallbacks.

References

1. Pasquale, G., Ciliberto, C., Rosasco, L. & Natale, L. Teaching robots visual perception through language and demonstration. *The Int. J. Robotics Res.* **38**, 1320–1341, DOI: 10.1177/0278364919863434 (2019).
2. He, K., Zhang, X., Ren, S. & Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778, DOI: 10.1109/CVPR.2016.90 (2016).
3. Ren, S., He, K., Girshick, R. & Sun, J. Faster r-cnn: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 91–99 (2015).
4. Redmon, J., Divvala, S., Girshick, R. & Farhadi, A. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 779–788, DOI: 10.1109/CVPR.2016.91 (2016).
5. He, K., Gkioxari, G., Dollár, P. & Girshick, R. Mask r-cnn. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2961–2969, DOI: 10.1109/ICCV.2017.322 (2017).
6. Howard, A. G. *et al.* Mobilenets: Efficient convolutional neural networks for mobile vision applications. In *arXiv preprint arXiv:1704.04861* (2017).
7. Dosovitskiy, A. *et al.* An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)* (2021).
8. del Solar, J. R., Verschae, R. & Correa, M. A survey on deep learning techniques for robot vision. *IEEE Transactions on Syst. Man, Cybern. Syst.* **50**, 1–14, DOI: 10.1109/TSMC.2018.2860335 (2020).

6 Appendix

6.1 resnet18

```
"""
cnn_resnet18_icub
"""

# ===== Imports =====
from pathlib import Path
import itertools
import time
import copy
import csv
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, classification_report
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, Subset
from torchvision import datasets, transforms, models

# ===== Global configuration =====
DATA_ROOT = Path("/Users/delaynomore/Downloads/iCubWorld1.0")
DOMAINS = ["human", "robot"]

# Manual hyperparameter grid
HYPERPARAM_GRID = {
    "batch_size": [16, 32],
    "learning_rate": [1e-3, 5e-4],
    "weight_decay": [0.0, 1e-4],
    "num_epochs": [15],
}

RESULTS_CSV = "cnn_results_icub.csv"
FIG_DIR = Path("figures_cnn")
FIG_DIR.mkdir(exist_ok=True)


def get_device():
    if torch.backends.mps.is_available():
        return torch.device("mps")
    if torch.cuda.is_available():
        return torch.device("cuda")
    return torch.device("cpu")


DEVICE = get_device()


# ===== Data loading =====
def create_transforms():

    train_transform = transforms.Compose([
        transforms.Resize(256),
```

```
        transforms.RandomResizedCrop(224),
        transforms.RandomHorizontalFlip(),
        transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
    ])
    eval_transform = transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
    ])
    return train_transform, eval_transform


def create_datasets_and_loaders(domain, batch_size):
    train_transform, eval_transform = create_transforms()
    train_dir = DATA_ROOT / domain / "train"
    test_dir = DATA_ROOT / domain / "test"

    full_train_dataset_aug = datasets.ImageFolder(train_dir, transform=train_transform)
    full_train_dataset_eval = datasets.ImageFolder(train_dir, transform=eval_transform)
# For Validation

    num_train = len(full_train_dataset_aug)
    indices = np.random.permutation(num_train)
    split = int(0.8 * num_train)
    train_indices, val_indices = indices[:split], indices[split:]

    train_dataset = Subset(full_train_dataset_aug, train_indices)
    val_dataset = Subset(full_train_dataset_eval, val_indices)
    test_dataset = datasets.ImageFolder(test_dir, transform=eval_transform)

    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True, num_workers
    val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False, num_workers=2)
    test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False, num_workers=

    return train_loader, val_loader, test_loader, full_train_dataset_aug.classes


# ===== Model =====
def build_resnet18_model(num_classes):
    model = models.resnet18(weights=models.ResNet18_Weights.IMAGENET1K_V1)
    for param in model.parameters():
        param.requires_grad = False

    in_features = model.fc.in_features
    model.fc = nn.Sequential(
        nn.Dropout(p=0.5),
        nn.Linear(in_features, num_classes),
    )
    return model.to(DEVICE)


# ===== Visualization Helper =====
```

```python
def save_error_cases(model, loader, domain, run_id, class_names):

    model.eval()
    inputs_list, preds_list, labels_list = [], [], []

    # Grab one batch or iterate until we find errors
    with torch.no_grad():
        for inputs, labels in loader:
            inputs = inputs.to(DEVICE)
            outputs = model(inputs)
            _, preds = torch.max(outputs, 1)

            # Store data to find errors
            inputs_list.append(inputs.cpu())
            preds_list.append(preds.cpu())
            labels_list.append(labels.cpu())

            # Limit to checking first few batches to save time
            if len(inputs_list) * loader.batch_size > 200:
                break

    inputs = torch.cat(inputs_list)
    preds = torch.cat(preds_list)
    labels = torch.cat(labels_list)

    # Find indices where prediction != label
    errors = (preds != labels).nonzero(as_tuple=False).squeeze()

    if errors.numel() == 0:
        return  # No errors found

    # Denormalize for plotting
    inv_normalize = transforms.Compose([
        transforms.Normalize(mean=[0., 0., 0.], std=[1 / 0.229, 1 / 0.224, 1 / 0.225]),
        transforms.Normalize(mean=[-0.485, -0.456, -0.406], std=[1., 1., 1.]),
    ])

    # Plot top 5 errors
    num_to_plot = min(5, errors.numel())
    fig, axes = plt.subplots(1, num_to_plot, figsize=(15, 3))
    if num_to_plot == 1: axes = [axes]

    for i, idx in enumerate(errors[:num_to_plot]):
        img_tensor = inputs[idx]
        img = inv_normalize(img_tensor).permute(1, 2, 0).numpy()
        img = np.clip(img, 0, 1)

        true_cls = class_names[labels[idx].item()]
        pred_cls = class_names[preds[idx].item()]

        axes[i].imshow(img)
        axes[i].set_title(f"True: {true_cls}\nPred: {pred_cls}", color='red', fontsize=10)
        axes[i].axis('off')

    plt.suptitle(f"{domain} Run {run_id} - Error Cases")
```

```
    plt.tight_layout()
    plt.savefig(FIG_DIR / f"{domain}_run{run_id}_errors.png")
    plt.close()


# ===== Training =====
def train_one_model(domain, hyperparams, run_id):
    batch_size = hyperparams["batch_size"]
    lr = hyperparams["learning_rate"]
    wd = hyperparams["weight_decay"]
    num_epochs = hyperparams["num_epochs"]

    train_loader, val_loader, test_loader, class_names = create_datasets_and_loaders(domain,
    model = build_resnet18_model(len(class_names))

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.fc.parameters(), lr=lr, weight_decay=wd)

    best_wts = copy.deepcopy(model.state_dict())
    best_val_acc = 0.0
    history = {"train_loss": [], "val_loss": [], "train_acc": [], "val_acc": []}

    for epoch in range(num_epochs):
        print(f"[{domain}] Run {run_id} | Epoch {epoch + 1}/{num_epochs}")
        for phase in ["train", "val"]:
            if phase == "train":
                model.train()
                loader = train_loader
            else:
                model.eval()
                loader = val_loader

            running_loss = 0.0
            running_corrects = 0
            total = 0

            for inputs, labels in loader:
                inputs, labels = inputs.to(DEVICE), labels.to(DEVICE)
                optimizer.zero_grad()

                with torch.set_grad_enabled(phase == "train"):
                    outputs = model(inputs)
                    _, preds = torch.max(outputs, 1)
                    loss = criterion(outputs, labels)

                    if phase == "train":
                        loss.backward()
                        optimizer.step()

                running_loss += loss.item() * inputs.size(0)
                running_corrects += torch.sum(preds == labels.data).item()
                total += inputs.size(0)

            epoch_acc = running_corrects / total
            epoch_loss = running_loss / total
```

```
            history[f"{phase}_loss"].append(epoch_loss)
            history[f"{phase}_acc"].append(epoch_acc)

            if phase == "val" and epoch_acc > best_val_acc:
                best_val_acc = epoch_acc
                best_wts = copy.deepcopy(model.state_dict())

        print(f"■■Val■Acc:■{history['val_acc'][-1]:.4f}")

    # Load best model for testing
    model.load_state_dict(best_wts)
    plot_training_curves(domain, run_id, history)

    # Evaluate Test
    test_acc, cm, y_true, y_pred = evaluate_on_test(model, test_loader)
    plot_confusion_matrix(domain, run_id, cm, class_names)


    save_error_cases(model, test_loader, domain, run_id, class_names)

    return best_val_acc, test_acc


def evaluate_on_test(model, test_loader):
    model.eval()
    y_true, y_pred = [], []
    with torch.no_grad():
        for inputs, labels in test_loader:
            inputs, labels = inputs.to(DEVICE), labels.to(DEVICE)
            outputs = model(inputs)
            _, preds = torch.max(outputs, 1)
            y_true.extend(labels.cpu().tolist())
            y_pred.extend(preds.cpu().tolist())

    cm = confusion_matrix(y_true, y_pred)
    acc = np.mean(np.array(y_true) == np.array(y_pred))
    return acc, cm, y_true, y_pred


def plot_training_curves(domain, run_id, history):
    epochs = range(1, len(history["train_loss"]) + 1)
    # Loss
    plt.figure()
    plt.plot(epochs, history["train_loss"], label="Train")
    plt.plot(epochs, history["val_loss"], label="Val")
    plt.title(f"{domain}■Run■{run_id}■Loss")
    plt.legend()
    plt.savefig(FIG_DIR / f"{domain}_run{run_id}_loss.png")
    plt.close()
    # Acc
    plt.figure()
    plt.plot(epochs, history["train_acc"], label="Train")
    plt.plot(epochs, history["val_acc"], label="Val")
    plt.title(f"{domain}■Run■{run_id}■Accuracy")
```

```
    plt.legend()
    plt.savefig(FIG_DIR / f"{domain}_run{run_id}_acc.png")
    plt.close()


def plot_confusion_matrix(domain, run_id, cm, class_names):
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=class_names)
    fig, ax = plt.subplots(figsize=(6, 6))
    disp.plot(ax=ax, cmap="Blues", colorbar=False)
    plt.xticks(rotation=45, ha="right")
    plt.title(f"{domain} Run {run_id} CM")
    plt.tight_layout()
    plt.savefig(FIG_DIR / f"{domain}_run{run_id}_cm.png")
    plt.close()


def run_experiments():
    with open(RESULTS_CSV, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["domain", "run_id", "batch", "lr", "wd", "epochs", "best_val_acc", "

        run_id = 0
        for domain in DOMAINS:
            # Grid search
            for bs, lr, wd, ep in itertools.product(
                    HYPERPARAM_GRID["batch_size"],
                    HYPERPARAM_GRID["learning_rate"],
                    HYPERPARAM_GRID["weight_decay"],
                    HYPERPARAM_GRID["num_epochs"]
            ):
                run_id += 1
                params = {"batch_size": bs, "learning_rate": lr, "weight_decay": wd, "num_epo
                print(f"--- Domain: {domain} | Run: {run_id} ---")

                val_acc, test_acc = train_one_model(domain, params, run_id)

                writer.writerow([domain, run_id, bs, lr, wd, ep, val_acc, test_acc])
                f.flush()


if __name__ == "__main__":
    run_experiments()
```

6.2 SIFT + BoW + SVM/KNN

```
"""
bow_sift_svm_icub.py
Traditional CV pipeline: SIFT + BoW + SVM/KNN.

"""

from pathlib import Path
import csv
import numpy as np
import matplotlib.pyplot as plt
import cv2
```

```python
from sklearn.cluster import MiniBatchKMeans
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, accuracy_score, classif
from sklearn.model_selection import train_test_split
from torchvision import datasets

# ===== Global configuration =====
DATA_ROOT = Path("/Users/delaynomore/Downloads/iCubWorld1.0")
DOMAINS = ["human", "robot"]

# Hyperparameters to explore
PARAMS_GRID = {
    "vocab_size": [50, 100],
    "classifier": ["svm", "knn"],
    "C": [0.1, 1.0],  # Only for SVM
    "n_neighbors": [5, 10]  # Only for KNN
}

RESULTS_CSV = "bow_sift_results_icub.csv"
FIG_DIR = Path("figures_bow_sift")
FIG_DIR.mkdir(exist_ok=True)


# ===== Data utilities =====
def get_image_paths_and_labels(root_dir):
    dataset = datasets.ImageFolder(root_dir)
    paths = [s[0] for s in dataset.samples]
    labels = [s[1] for s in dataset.samples]
    return np.array(paths), np.array(labels), dataset.classes


# ===== SIFT & BoW =====
def compute_sift_descriptors(image_paths, max_features=500):
    sift = cv2.SIFT_create(nfeatures=max_features)
    desc_list = []
    for path in image_paths:
        img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
        if img is None:
            desc_list.append(None)
            continue
        _, descriptors = sift.detectAndCompute(img, None)
        desc_list.append(descriptors)
    return desc_list


def build_vocabulary(descriptors_list, vocab_size, max_samples=100000):
    all_desc = [d for d in descriptors_list if d is not None]
    if not all_desc: raise RuntimeError("No descriptors found.")

    # Stack and Force Float64 for Scikit-Learn compatibility
    all_desc = np.vstack(all_desc).astype(np.float64)

    if len(all_desc) > max_samples:
        idx = np.random.choice(len(all_desc), max_samples, replace=False)
```

```
        all_desc = all_desc[idx]

    print(f"■■Clustering■{len(all_desc)}■descriptors■->■{vocab_size}■words...")
    kmeans = MiniBatchKMeans(n_clusters=vocab_size, batch_size=1000, random_state=42, n_init=
    kmeans.fit(all_desc)
    return kmeans


def compute_bow_features(desc_list, kmeans, vocab_size):
    features = []
    for desc in desc_list:
        if desc is None:
            features.append(np.zeros(vocab_size))
            continue

        # FIX: Explicit cast to float64 before prediction
        words = kmeans.predict(desc.astype(np.float64))

        hist, _ = np.histogram(words, bins=np.arange(vocab_size + 1))
        norm = np.linalg.norm(hist)
        if norm > 0: hist = hist / norm  # L2 norm usually better for SVM
        features.append(hist)
    return np.stack(features)


# ===== Visualization =====
def save_error_cases(domain, run_id, X_test, y_test, y_pred, test_paths, class_names):

    errors = np.where(y_test != y_pred)[0]
    if len(errors) == 0: return

    num_plot = min(5, len(errors))
    fig, axes = plt.subplots(1, num_plot, figsize=(15, 3))
    if num_plot == 1: axes = [axes]

    for i, idx in enumerate(errors[:num_plot]):
        path = test_paths[idx]
        img = cv2.imread(path)
        if img is not None:
            img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

            true_lbl = class_names[y_test[idx]]
            pred_lbl = class_names[y_pred[idx]]

            axes[i].imshow(img)
            axes[i].set_title(f"T:■{true_lbl}\nP:■{pred_lbl}", color='red', fontsize=10)
            axes[i].axis('off')

    plt.suptitle(f"{domain}■Run■{run_id}■Errors")
    plt.tight_layout()
    plt.savefig(FIG_DIR / f"{domain}_run{run_id}_errors.png")
    plt.close()


# ===== Experiment Logic =====
```

```
def run_experiment_logic():
    with open(RESULTS_CSV, "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["domain", "run_id", "vocab", "clf_type", "param", "val_acc", "test_a

        run_id = 0
        for domain in DOMAINS:
            print(f"\nProcessing Domain: {domain}")

            # 1. Load Data
            train_full_paths, train_full_labels, classes = get_image_paths_and_labels(DATA_RO
            test_paths, test_labels, _ = get_image_paths_and_labels(DATA_ROOT / domain / "tes

            # 2. Split Train into Train/Val (Essential for hyperparam tuning)
            train_paths, val_paths, train_y, val_y = train_test_split(
                train_full_paths, train_full_labels, test_size=0.2, stratify=train_full_labels
            )

            # 3. Compute Descriptors (expensive step, do once per split)
            print("  Computing SIFT descriptors...")
            train_desc = compute_sift_descriptors(train_paths)
            val_desc = compute_sift_descriptors(val_paths)
            test_desc = compute_sift_descriptors(test_paths)

            # 4. Iterate Vocab Size
            for vocab in PARAMS_GRID["vocab_size"]:
                # Build Vocab on TRAIN only
                kmeans = build_vocabulary(train_desc, vocab)

                # Featurize
                X_train = compute_bow_features(train_desc, kmeans, vocab)
                X_val = compute_bow_features(val_desc, kmeans, vocab)
                X_test = compute_bow_features(test_desc, kmeans, vocab)

                # 5. Iterate Classifiers
                for clf_type in PARAMS_GRID["classifier"]:
                    # Specific params for clf
                    param_list = PARAMS_GRID["C"] if clf_type == "svm" else PARAMS_GRID["n_ne

                    for p in param_list:
                        run_id += 1
                        print(f"  Run {run_id}: Vocab={vocab}, {clf_type}={p}")

                        if clf_type == "svm":
                            clf = SVC(kernel='linear', C=p)
                        else:
                            clf = KNeighborsClassifier(n_neighbors=p)

                        # Train
                        clf.fit(X_train, train_y)

                        # Validate (Decision metric)
                        val_pred = clf.predict(X_val)
                        val_acc = accuracy_score(val_y, val_pred)
```

```
                    # Test (Reporting metric - strictly only for final table)
                    test_pred = clf.predict(X_test)
                    test_acc = accuracy_score(test_labels, test_pred)

                    writer.writerow([domain, run_id, vocab, clf_type, p, val_acc, test_ac
                    f.flush()

                    # Save CM and Errors for the report
                    cm = confusion_matrix(test_labels, test_pred)
                    disp = ConfusionMatrixDisplay(cm, display_labels=classes)
                    fig, ax = plt.subplots(figsize=(6, 6))
                    disp.plot(ax=ax, cmap="Purples", colorbar=False)
                    plt.title(f"{domain}■{clf_type}({p})■V={vocab}")
                    plt.savefig(FIG_DIR / f"{domain}_run{run_id}_cm.png")
                    plt.close()

                    save_error_cases(domain, run_id, X_test, test_labels, test_pred, test


if __name__ == "__main__":
    run_experiment_logic()
```